04

Part 4 · Networking

How cloud resources are addressed, secured and connected: IPs, DNS, subnets and load balancing.

- Public vs Private IP
- DNS
- HTTP vs HTTPS · Ports
- Firewalls
- VPC & Subnets
- Connecting networks
- Load Balancer

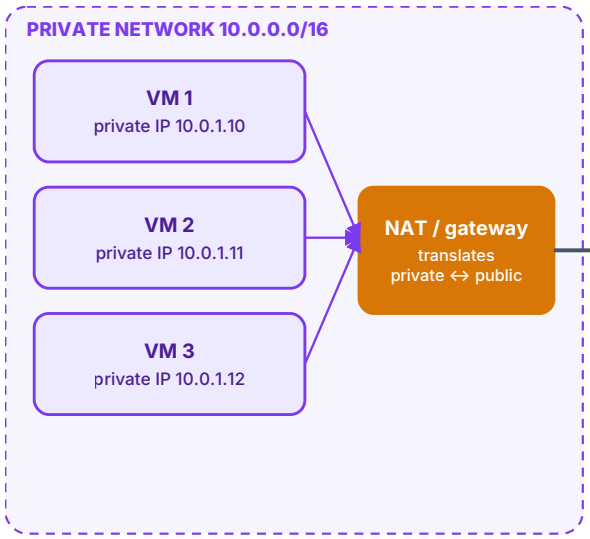
4.1 Public IP vs Private IP

★★★

IN ONE LINE

Public addresses are reachable on the internet; private addresses live only inside a network.

Every device on a network needs an **IP address**. **Public IPs** are unique across the whole internet and reachable from outside. **Private IPs** are used only inside a private network and are **not routable on the internet**; many networks reuse the same private ranges.



Private ranges	Block	Addresses
Class A	10.0.0.0/8	16.7 million
Class B	172.16.0.0/12	1 million
Class C	192.168.0.0/16	65,536

Figure 4.1 — Private addresses inside, a public address outside, NAT in between

	Public IP	Private IP
Reachable from internet?	Yes (if firewall allows)	No, only inside the network or via VPN / NAT
Uniqueness	Globally unique	Unique only within its network
Assigned by	ISP / cloud provider	You (network admin / DHCP)
Cost / risk	May cost money; exposed to attacks	Free; safer by default
Typical use	Web servers, load balancers, NAT gateways	App servers, databases, internal services
Cloud notes	**Elastic / static IP** stays the same when VM restarts	Fixed in the subnet CIDR

IPv4 vs IPv6

IPv4: 32-bit, e.g. `192.168.1.10`, about 4.3 billion addresses (exhausted). **IPv6**: 128-bit, e.g. `2001:db8::1`, effectively unlimited.

NAT in one line

Network Address Translation lets many private machines **share one public IP** to reach the internet, while outsiders cannot start connections to them.

INTERVIEW ANGLE

Best practice: only **load balancers and a few entry points** get public IPs. Application servers and databases stay on **private IPs** in private subnets (see 4.6).

4.2 DNS (Domain Name System)

★★★

IN ONE LINE

The internet's phone book: translates domain names into IP addresses.

DNS (Domain Name System) is the internet's phone book: it translates human-friendly **domain names** (`www.example.com`) into **IP addresses** (`93.184.216.34`) that computers use to connect.

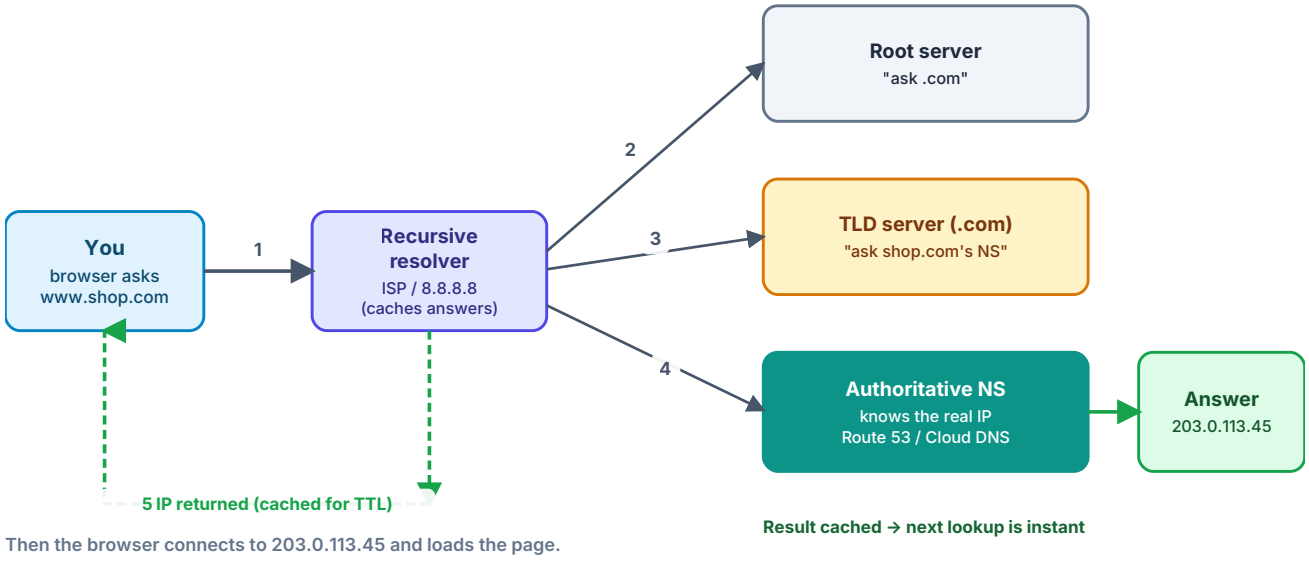


Figure 4.2 — Resolving a domain name, step by step

Record	Purpose	Example
A	Name → IPv4 address	`shop.com → 203.0.113.45`
AAAA	Name → IPv6 address	`shop.com → 2001:db8::1`
CNAME	Alias of another name	`www.shop.com → shop.com`
MX	Mail servers for the domain	`shop.com → mail.shop.com`
TXT	Free text: domain verification, SPF	`"v=spf1 ..."`
NS	Which servers are authoritative	`ns1.route53...`

TTL (time to live)

How long a resolver may cache an answer. Short TTL = changes spread fast; long TTL = fewer lookups. Lower TTL *before* a planned migration.

Cloud DNS services

Amazon Route 53, **Azure DNS**, **Cloud DNS**. They also offer **health-check-based routing**, latency-based and geo routing — used for failover and global load distribution.

INTERVIEW ANGLE

If a site "works by IP but not by name", the problem is DNS. Useful commands: `nslookup example.com` , `dig example.com` .

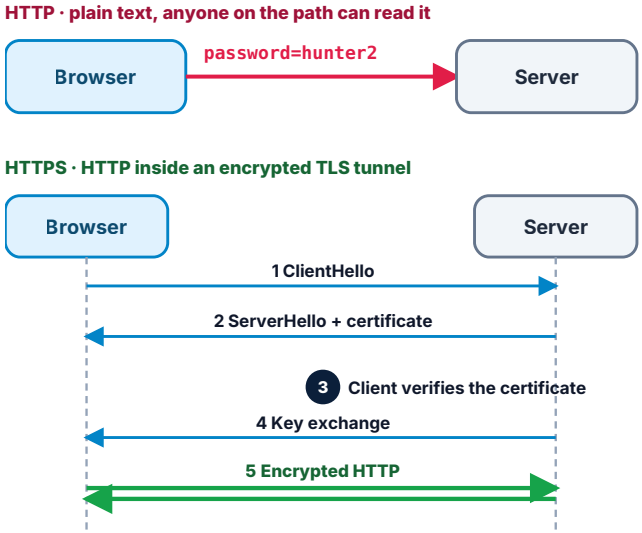
4.3 HTTP vs HTTPS

★★★

IN ONE LINE

HTTP carries web traffic in plain text; HTTPS wraps it in TLS encryption with verified identity.

HTTP (HyperText Transfer Protocol) is the language browsers and servers use to exchange web content. **HTTPS** is HTTP wrapped in **TLS encryption**, giving confidentiality, integrity and proof of the server's identity.



	HTTP	HTTPS
Port	80	443
Encrypted	No	Yes (TLS)
Tamper-proof	No	Yes
Server identity	Unverified	Certificate
Cost	free	free certs (ACM, Let's Enc

Figure 4.3a — HTTP is readable on the wire; HTTPS first sets up an encrypted tunnel

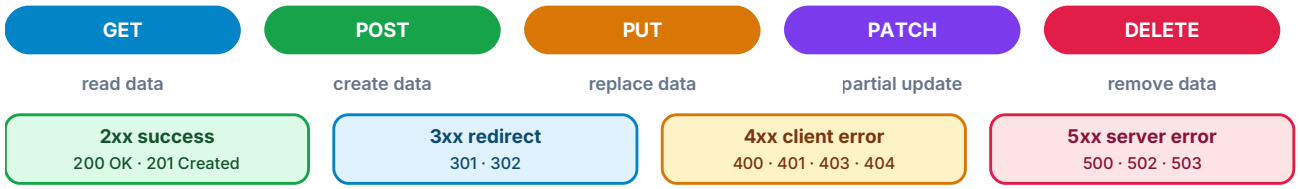


Figure 4.3b — Common HTTP methods and status code families

What a certificate proves

A trusted **Certificate Authority (CA)** signed a statement that this public key belongs to this domain. Browsers check the signature, domain and expiry before trusting the site.

In the cloud

Terminate HTTPS at the **load balancer / CDN** using managed certificates (**AWS ACM**, Azure Key Vault certs). Redirect HTTP to HTTPS. Use HTTPS for every API, even internal ones where possible.

COMMON TRAP

401 means "not authenticated" (who are you?), **403** means "authenticated but not allowed". **502/503/504** usually point at the server or a gateway in front of it. Interviewers use status codes to test debugging skills.

4.4 Ports and Protocols

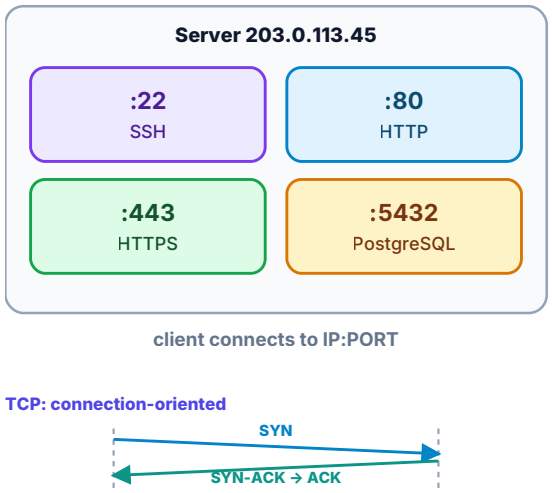
★★★

IN ONE LINE

Ports identify programs on a machine; protocols define how two systems communicate.

An IP address finds the **machine**; a **port** (0–65535) finds the **program** on it. A **protocol** is an agreed set of rules for how two sides talk (HTTP, DNS, SSH…). Firewalls (next topic) work by allowing or blocking **protocol + port + source**.

IP ADDRESS = the building · PORT = the apartment · PROTOCOL = the language spoken



Port	Service	Used for
22	SSH	Secure remote login to Linux
80	HTTP	Unencrypted web
443	HTTPS	Encrypted web, APIs
3389	RDP	Windows remote desktop
53	DNS	Name resolution (UDP/TCP)
3306 / 543	MySQL / Postgres	Relational databases
1433	SQL Server	Microsoft databases
6379	Redis	In-memory cache
9092	Kafka	Message broker
25 · 587	SMTP	Sending email

Figure 4.4 — IP + port + protocol, and the commonly asked port numbers

	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless, fire and forget
Reliability	Guaranteed, ordered delivery; retransmits	No guarantee, no ordering
Speed	Slower (overhead)	Faster, lightweight
Used by	HTTP/HTTPS, SSH, databases, email	DNS queries, video streaming, VoIP, gaming

Port ranges

- **0–1023** well-known (HTTP, SSH…)
- **1024–49151** registered applications
- **49152–65535** ephemeral (client side)

Other protocols to recognise

- **FTP/SFTP** file transfer (21 / 22)
- **SMTP/IMAP** email
- **ICMP** ping
- **DHCP** hands out IP addresses
- **gRPC, WebSocket, MQTT** modern app / IoT

INTERVIEW ANGLE

Memorise **22, 80, 443, 3389, 53** and the database ports (**3306, 5432, 1433**). A very common scenario: “I cannot SSH into my VM” → check the firewall / security group allows **TCP 22** from your IP.

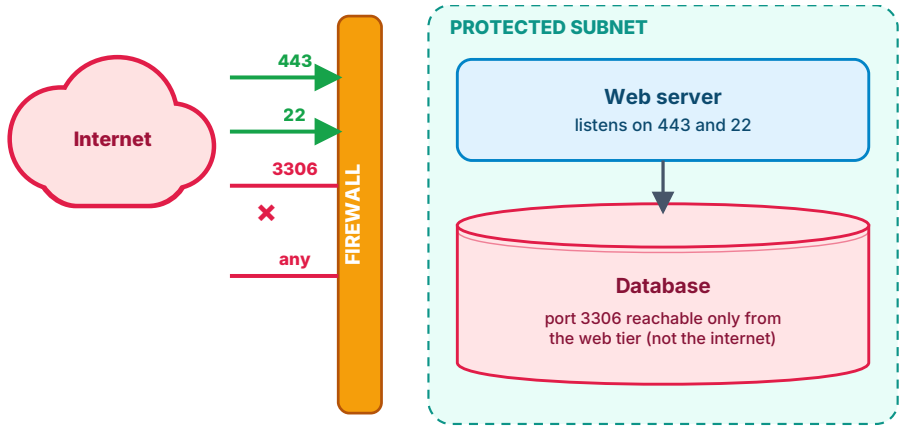
4.5 Firewall

★★★

IN ONE LINE

Rules that allow or block network traffic by source, destination, protocol and port.

A **firewall** filters network traffic based on rules — allowing or blocking by **source, destination, protocol and port**. It is the network's security guard. In the cloud, firewalls are virtual and attached to resources or subnets.



Rule	Value
Direction	Inbound
Protocol	TCP
Port	443
Source	0.0.0.0/0
Action	ALLOW

Default deny: anything not explicitly allowed is blocked.

Rules match on direction · protocol · port · source / destination IP range.

Figure 4.5 — A firewall allows only what the rules permit

	Security group (SG) / NSG	Network ACL (NACL)
Applies to	A VM / network interface (instance level)	A whole subnet
State	**Stateful**: reply traffic is automatically allowed	**Stateless**: must allow both directions explicitly
Rules	Allow rules only (everything else denied)	Allow *and* deny rules, evaluated in order
Typical use	Primary day-to-day control	Extra subnet-level guardrail, blocking an IP range

Kinds of firewall

- **Network firewall** — layers 3–4 (IP, port).
- **WAF** (web application firewall) — layer 7, blocks SQL injection, XSS, bots.
- **Next-gen firewall** — deep packet inspection, intrusion prevention.

Good habits

- Default deny; open only what is needed.
- Never open **22 / 3389 / DB ports to 0.0.0.0/0**.
- Reference other security groups instead of IP lists.
- Separate tiers: web → app → database.

COMMON TRAP

`0.0.0.0/0` means "the entire internet". Seeing it on an SSH, RDP or database rule is a red flag in any security review.

4.6 Virtual Network and Subnet

★★★

IN ONE LINE

Your private, isolated cloud network, divided into subnets with their own routing and exposure.

A **virtual network** (AWS **VPC**, Azure **VNet**, GCP **VPC**) is your own logically isolated network in the cloud, where you choose the IP range, subnets, routes and firewalls. A **subnet** is a smaller slice of that range, tied to one Availability Zone, used to group resources with similar exposure.

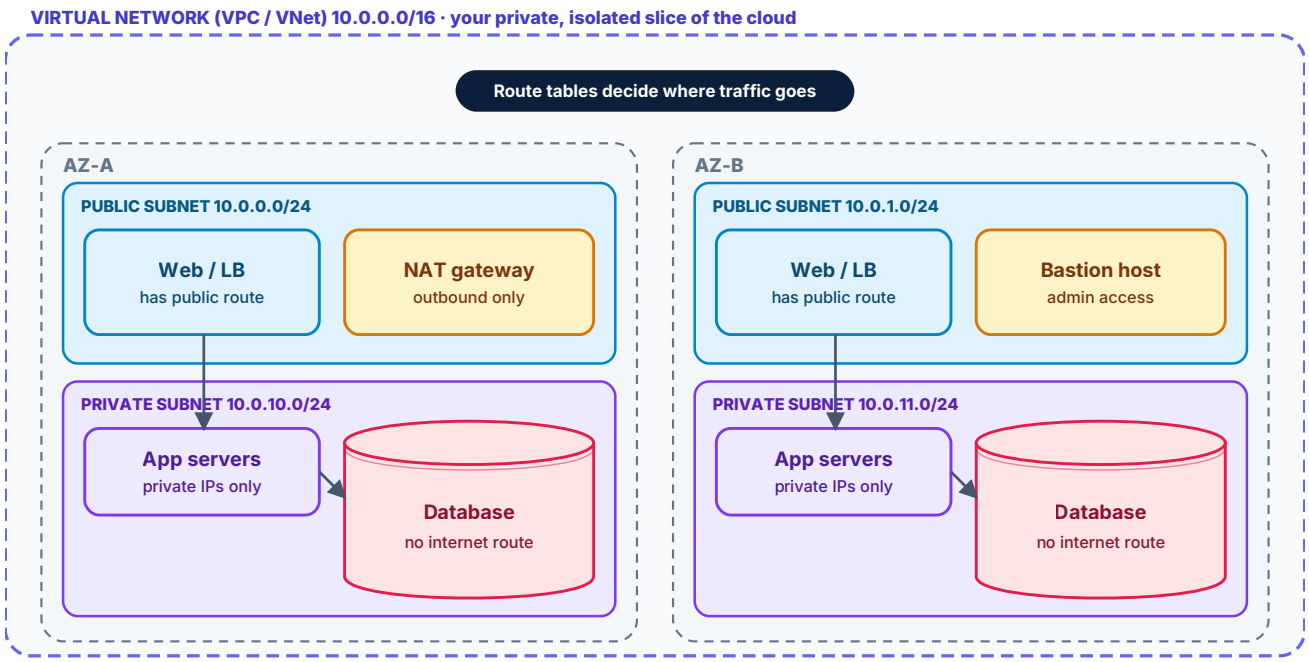


Figure 4.6a — A typical VPC: public and private subnets across two AZs

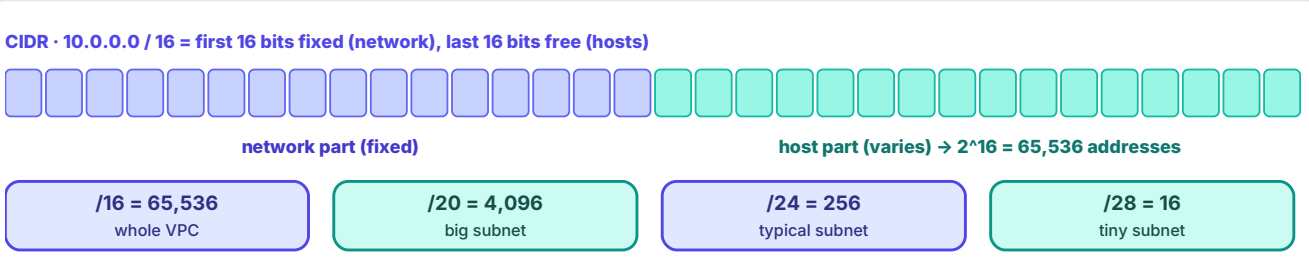


Figure 4.6b — CIDR notation sizes the network

Component	Role
Public subnet	Has a route to the Internet Gateway; holds load balancers, bastion hosts, NAT gateways.
Private subnet	No direct internet route; holds app servers and **databases**.
Internet Gateway	The door between the VPC and the public internet.
NAT Gateway	Lets private instances start *outbound* connections (updates, APIs) without being reachable inbound.
Route table	Rules deciding where each destination range is sent.
Security group / NACL	Firewalls for instances and subnets (4.5).

INTERVIEW ANGLE

The textbook secure design: **public subnet = load balancer only; private subnets = app and database**. Plan CIDR blocks that **do not overlap** with other VPCs or your on-prem network, otherwise you cannot connect them later.

4.7 Connecting Networks: VPN, Peering & Private Links

★★☆

IN ONE LINE

Linking data centres, VPCs and cloud services securely.

Real systems rarely live in one network. You connect a data centre to the cloud, one VPC to another, and private resources to cloud services — without exposing them to the open internet.

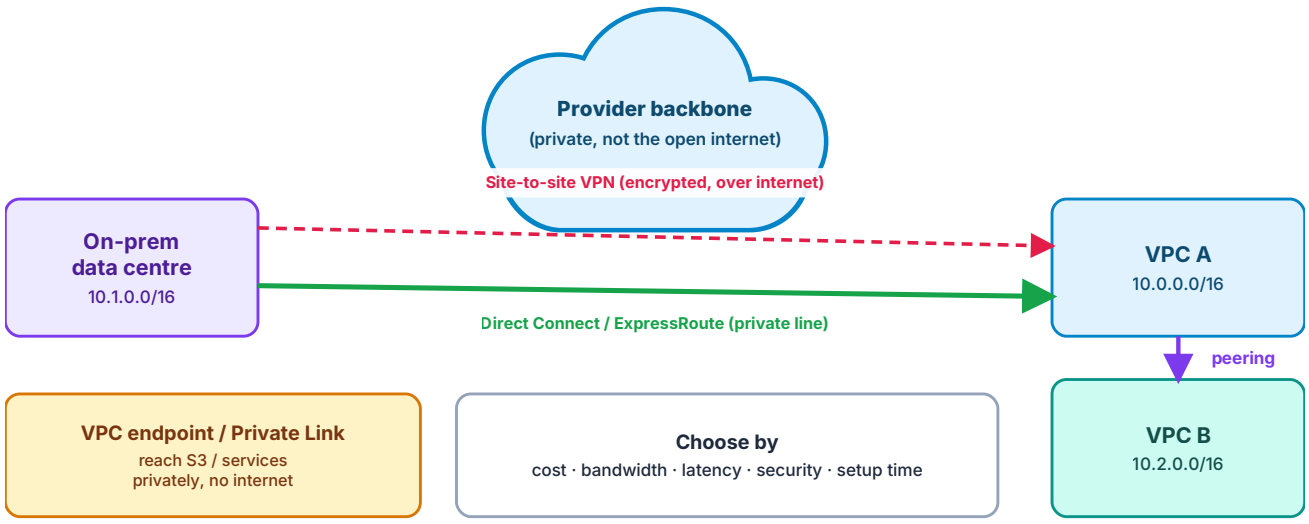


Figure 4.7 — Ways to connect networks

Option	What it is	Pros	Cons
Site-to-site VPN	Encrypted tunnel over the public internet	Quick, cheap	Variable latency, limited bandwidth
Direct Connect / ExpressRoute / Interconnect	Dedicated private line to the provider	Stable, fast, predictable	Costly, weeks to install
VPC peering	Private link between two VPCs	Simple, low latency	Not transitive; no overlapping CIDR
Transit gateway / hub	Central router for many networks	Scales to many VPCs	Extra cost
Private endpoint (PrivateLink)	Reach a service via a private IP	Traffic never leaves the backbone	One endpoint per service

Bastion host / jump box

A hardened VM in the public subnet used as the only entry point for admin access to private machines. Modern alternatives: **SSM Session Manager**, Azure Bastion.

VPN vs proxy (quick)

A **VPN** encrypts and tunnels all traffic between networks or a user and a network. A **proxy** relays specific application requests on your behalf.

DATA ENGINEERING TIE-IN

Connecting an on-prem database to a cloud data platform for ingestion normally uses a VPN or Direct Connect plus a private endpoint to the object store, so data never crosses the public internet.

4.8 Load Balancer

★★★

IN ONE LINE

Spreads traffic across healthy servers behind a single entry point.

A **load balancer** distributes incoming requests across a group of servers so no single one is overloaded, and sends traffic only to servers that pass **health checks**. It gives clients **one stable address** while servers come and go behind it.

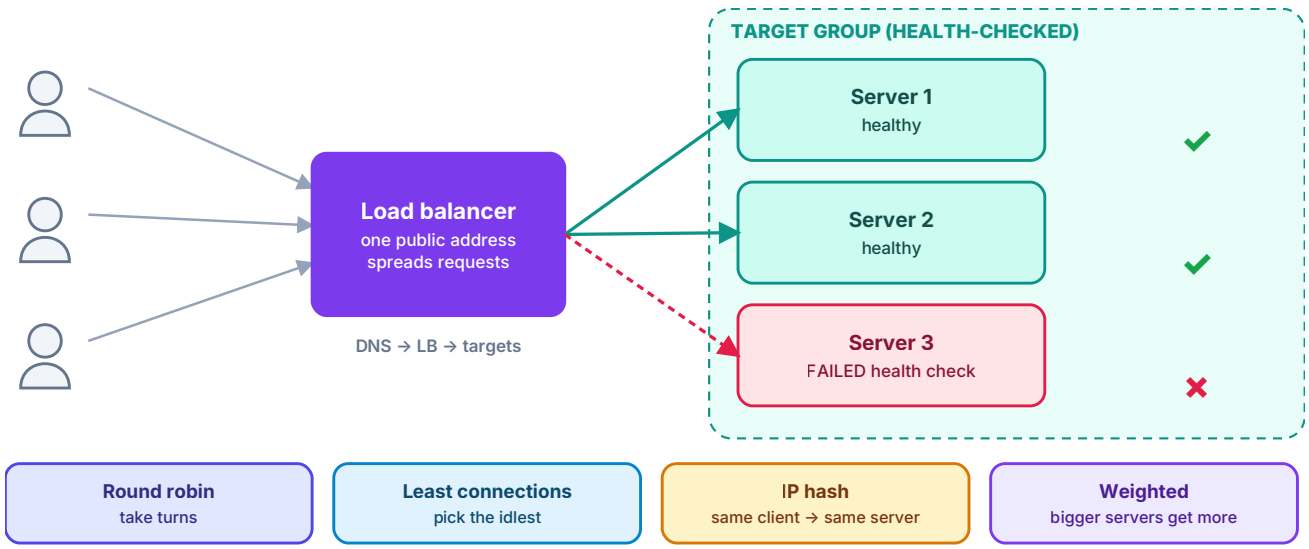


Figure 4.8 — One front door, several health-checked servers

	Layer 4 (network / TCP)	Layer 7 (application / HTTP)
Sees	IPs and ports only	URLs, headers, cookies, methods
Speed	Very fast, ultra-low latency	Slightly slower, smarter
Features	Pass-through TCP/UDP, static IPs	Path routing (`/api` vs `/img`), host routing, TLS termination, WAF
Examples	AWS NLB, Azure Load Balancer	AWS ALB, Azure Application Gateway, GCP HTTP(S) LB

What it gives you

- **Scalability** — add servers behind it.
- **High availability** — dead servers removed automatically.
- **Security** — servers stay private; TLS ends here.
- **Zero-downtime deploys** — drain, update, return.

Related ideas

- **Sticky sessions** keep a user on one server (avoid if you can).
- **Global load balancing** across regions (DNS / anycast).
- The LB itself is redundant — managed, multi-AZ.

INTERVIEW ANGLE

Load balancer + auto scaling group across two AZs is the **standard answer** to “how do you make a web app scalable and highly available?”. Make the servers **stateless**: keep sessions in a cache or database, not in server memory.